

## Punto 3.12 — Buenas prácticas en el acceso a datos y composición de consultas

### Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.11. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

### Objetivo práctico

Implementar y ejecutar **Buenas prácticas en el acceso a datos y composición de consultas** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

### Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.12
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

### Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

### Paso 3: Revisar los cambios de este punto

Los elementos añadidos en este estado son: `ObtenerResumenesPendientesOptimizado`, `ExisteAlgunaOrdenPendiente`, `ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano` y `ObtenerPorNumeroOptimizado`. Como cierre de arquitectura se retiran `Consulta` y `ObtenerSqlFundamentos` del puerto público; al retirar `AutoInclude` como comportamiento global también desaparecen `ObtenerOrdenesAutoInclude` y `ObtenerOrdenesIgnorandoAutoInclude`. La composición queda encapsulada en `Infraestructure`. El `AsSplitQuery()` final se conserva sobre dos colecciones hermanas —`Planchas` y `OrdenesAleaciones`— más `Detalle`.

### Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/BuenasPracticasUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class BuenasPracticasUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public BuenasPracticasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===");
        var resúmenes = _unidad.Ordenes.ObtenerResúmenesPendientesOptimizado();
        var existe = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();
        var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();
        var inexistente = _unidad.Ordenes.ObtenerPorNumeroOptimizado("OF-2024-9999");
        if (resúmenes.Count != 3 || !existe || split.Count != 5 || inexistente is not null) throw new
        InvalidOperationException("Buenas prácticas: validación E2E fallida.");
        Console.WriteLine($"Pendientes proyectadas: {resúmenes.Count} | Any: {existe} | SplitQuery:
        {split.Count}");
        Console.WriteLine("El contrato final ya no expone IQueryable fuera de Infrastructure.");
    }
}

```

### Explicación línea a línea del caso de uso 3.12

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial `IUnidadDeTrabajo` e `IOrdenRepositorio`, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class BuenasPracticasUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de `DbContext`.

Línea 8: `public BuenasPracticasUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== BUENAS PRÁCTICAS EN EL ACCESO A DATOS ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var resúmenes = _unidad.Ordenes.ObtenerResúmenesPendientesOptimizado();` → Ejecuta una lectura proyectada y no-tracking de las órdenes pendientes, sin exponer `IQueryable` al caso de uso.

Línea 14: `var existe = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();` → Ejecuta `Any` para expresar directamente la comprobación de existencia de órdenes pendientes.

Línea 15: `var split = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();` → Carga el grafo final con `AsSplitQuery` sobre dos colecciones hermanas y una referencia para controlar la multiplicación de filas.

Línea 16: `var inexistente = _unidad.Ordenes.ObtenerPorNumeroOptimizado("OF-2024-9999");` → Ejecuta una búsqueda de solo lectura con `FirstOrDefault` y valida explícitamente el caso sin resultado.

Línea 17: `if (resumenes.Count != 3 || !existe || split.Count != 5 || inexistente is not null)`  
`throw new InvalidOperationException("Buenas prácticas: validación E2E fallida.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 18: `Console.WriteLine($"Pendientes proyectadas: {resumenes.Count} | Any: {existe} | SplitQuery: {split.Count}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 19: `Console.WriteLine("El contrato final ya no expone IQueryable fuera de Infrastructure.");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

## | Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<BuenasPracticasUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.12 OK");
```

### Explicación línea a línea de Program.cs 3.12

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa `AceriaDbContext` para preparar la base de demostración desde el `composition root`.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por `Migrate`, `ChangeTracker` y las operaciones de consulta del `checkpoint`.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa `ConfigurationBuilder` y las extensiones necesarias para cargar `appsettings.json` y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa `ServiceCollection`, `AddScoped`, `CreateScope` y `GetRequiredService` para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar `appsettings.json`.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<BuenasPracticasUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<BuenasPracticasUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.12 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

## Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

## Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.12 OK
```

## Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.12, `IOrdenRepositorio` ya no devuelve `IQueryable<OrdenFabricacion>`; el objetivo es cerrar el módulo con una frontera limpia entre Application e Infrastructure.

## Paso 9: Diagnosticar un error común

**Error:** Exponer `IQueryable` desde Application deja que capas superiores acoplen su lógica al proveedor; el contrato final lo elimina.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

### Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

**Archivo:** `src/AceriaData.Application/Interfaces.cs`

**Marcador:** `ERROR CONTROLADO M03 3.12 - PUERTO IQUERYABLE ANTIPATRON`

```
// Se activa únicamente en la copia desechable del Paso 9.
IQueryable<OrdenFabricacion> ConsultaAntiPatron();
```

**Archivo:** src/AceriaData.Infrastructure/Repositories/Repositories.cs

**Marcador:** ERROR CONTROLADO M03 3.12 - IMPLEMENTACION IQUERYABLE ANTIPATRON

```
// Expone el proveedor a Application para demostrar por contraste por qué el contrato final lo retira.
public IQueryable<OrdenFabricacion> ConsultaAntiPatron() =>
    _context.OrdenesFabricacion.AsNoTracking();
```

**Archivo:** src/AceriaData.Application/BuenasPracticasUseCase.cs

**Marcador:** ERROR CONTROLADO M03 3.12 - IQUERYABLE EXPUESTO EN APPLICATION

```
// Demuestra el Paso 9: compila y funciona, pero permite que Application componga
// directamente la consulta del proveedor y rompe la frontera buscada al cerrar M3.
var antiPatron = _unidad.Ordenes.ConsultaAntiPatron()
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .ToList();

if (antiPatron.Count != 3)
    throw new InvalidOperationException("Error controlado 3.12: IQueryable público devolvió un
    resultado inesperado.");

Console.WriteLine($"Error controlado 3.12 OK | IQueryable compuesto en Application:
    {antiPatron.Count}");
```

**Comprobación esperada:** exponer IQueryable desde Application acopla la capa al proveedor y rompe la frontera final.

## Laboratorio adicional del punto 3.12

### Diagnóstico técnico

El cierre conserva `AsNoTracking`, proyecciones, `Any`, `FirstOrDefault` y `SplitQuery` cuando el shape lo justifica. Retira `IQueryable` del puerto y `AutoInclude` del modelo para evitar decisiones implícitas en Application.

### Reto resuelto y verificación adicional

Valida tres pendientes proyectadas, `Any == true`, cinco órdenes en la consulta `SplitQuery` y `null` para `0F-2024-9999`. El `SplitQuery` final carga Planchas, `OrdenesAleaciones→Aleacion` y Detalle.

### Errores comunes revisados

| Error                                 | Causa                                                                | Solución                                                       |
|---------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------|
| AsNoTracking en consulta de escritura | Se aplicó AsNoTracking a una consulta que modifica datos             | Usar AsNoTracking solo en consultas de solo lectura            |
| Count() > 0 en lugar de Any           | Se usó Count para saber si hay elementos                             | Usar Any                                                       |
| First en lugar de FirstOrDefault      | Se usó First cuando puede no haber resultados                        | Usar FirstOrDefault                                            |
| Multiplicación de filas               | Se cargaron colecciones hermanas al mismo nivel en modo single-query | Comparar AsSingleQuery() y AsSplitQuery() para el volumen real |
| Materialización prematura             | Se llamó a ToList antes de aplicar todos los filtros                 | Materializar solo al final                                     |

| Error                  | Causa                                                         | Solución                                                               |
|------------------------|---------------------------------------------------------------|------------------------------------------------------------------------|
| N+1                    | Se accede en un bucle a navegaciones Lazy todavía no cargadas | Elegir de forma explícita Eager Loading, proyección o Explicit Loading |
| Falta de documentación | No se documentaron las decisiones                             | Añadir comentarios XML                                                 |

## Analogía operativa

Las buenas prácticas de acceso a datos en una acería son como las normas de seguridad y eficiencia de la planta. No se trata solo de producir acero, sino de producirlo de forma segura, eficiente y sostenible. Usar proyecciones es como pedir solo los datos que se necesitan: no se pide la carpeta completa si solo se necesita el número de orden. Usar AsNoTracking es como no registrar cada plancha en el libro de producción si solo se va a consultar. Usar Any es como comprobar si hay planchas en el almacén sin contarlas todas. Usar FirstOrDefault es como buscar una plancha por su número y devolver null si no existe, en lugar de lanzar una alarma. Usar AsSplitQuery es como dividir una orden de búsqueda grande en varias más pequeñas para evitar un envío masivo. Documentar las decisiones es como escribir las normas en el manual de la planta para que todos las conozcan. Así funcionan las buenas prácticas en EF Core: producen código más eficiente, más mantenible y más predecible.

## Paso 10: Reto resuelto y conexión con el siguiente punto

**Reto:** Revisa IOrdenRepositorio y justifica por qué ya no contiene IQueryable aunque Infrastructure siga componiendo consultas.

### Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

**Archivo:** src/AceriaData.Application/BuenasPracticasUseCase.cs

**Marcador:** RETO M03 3.12 - FRONTERA LIMPIA Y BUENAS PRACTICAS

```
// La validación ejecutable usa únicamente métodos específicos del puerto.
// El E2E comprobará además que el contrato activo no contiene IQueryable
// aunque BuscarOrdenes siga componiendo IQueryable dentro de Infrastructure.
var retoResumenes = _unidad.Ordenes.ObtenerResumenesPendientesOptimizado();
var retoExiste = _unidad.Ordenes.ExisteAlgunaOrdenPendiente();
var retoSplit = _unidad.Ordenes.ObtenerOrdenesConPlanchasYDetalleSinProductoCartesiano();
var retoInexistente = _unidad.Ordenes.ObtenerPorNumeroOptimizado("0F-2024-9999");

if (retoResumenes.Count != 3 || !retoExiste || retoSplit.Count != 5 || retoInexistente is not
null)
    throw new InvalidOperationException("Reto 3.12: la frontera final no conserva el
comportamiento esperado.");

Console.WriteLine($"Reto 3.12 OK | Pendientes: {retoResumenes.Count} | Any: {retoExiste} |
Split: {retoSplit.Count} | Inexistente: null");
```

**Comprobación esperada:** la frontera limpia conserva las optimizaciones dentro de Infrastructure sin devolver IQueryable.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar `3.12 OK`, el siguiente estado parte exactamente de esta solución; con ello queda cerrado el Módulo 3.